

A Technical Introduction to Transformer architecture deep learning

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Section 1

/ second sublayer */* $z_e_copy \leftarrow \text{copy}(z_e)$ for each t in $1:\text{length}(z_e)$ do $z_e[t] \leftarrow \text{layer.layer_norm}(z_e[t])$ $z_e \leftarrow \text{layer.feedforward}(z_e)$ for each t in $1:\text{length}(z_e)$ do $z_e[t] \leftarrow z_e[t] + z_e_copy[t]$

Section 2

$f(t) = (e^{it/rk})_{k=0,1,\dots,d/2-1}$ $\{ \text{displaystyle } f(t) = \left(e^{it/r^k} \right)_{k=0,1,\dots,d/2-1} \}$ where $r = N^{2/d}$ $\{ \text{displaystyle } r = N^{2/d} \}$. The main reason for using this positional encoding function is that using it, shifts are linear transformations: $f(t + \Delta t) = \text{diag}(f(\Delta t)) f(t)$ $\{ \text{displaystyle } f(t + \Delta t) = \text{diag}(f(\Delta t)) f(t) \}$ where $\Delta t \in \mathbb{R}$ $\{ \text{displaystyle } \Delta t \in \mathbb{R} \}$ is the distance one wishes to shift. This allows the transformer to take any encoded position, and find the encoding of the position n -steps-ahead or n -steps-behind, by a matrix multiplication. By taking a linear sum, any convolution can also be implemented as linear transformations: $\sum_j c_j f(t + \Delta t_j) = (\sum_j c_j \text{diag}(f(\Delta t_j))) f(t)$ $\{ \text{displaystyle } \sum_j c_j f(t + \Delta t_j) = \left(\sum_j c_j \right) \text{diag}(f(\Delta t_j)) f(t) \}$ for any constants c_j $\{ \text{displaystyle } c_j \}$. This allows the transformer to take any encoded position and find a linear sum of the encoded locations of its neighbors. This sum of encoded positions, when fed into the attention mechanism, would create attention weights on its neighbors, much like what happens in a convolutional neural network language model. In the author's words, "we hypothesized it would allow the model to easily learn to attend by relative position." In typical implementations, all operations are done over the real numbers, not the complex numbers, but since complex multiplication can be implemented as real 2-by-2 matrix multiplication, this is a mere notational difference.

Section 3

$\text{MultiheadAttention}(Q, K, V) = \text{Concat}_{i \in [n \text{ heads}]} (\text{Attention}(XW_i^Q, XW_i^K, XW_i^V)) W^O$ $\{ \text{displaystyle } \text{MultiheadAttention}(Q, K, V) = \left(\text{Concat}_{i \in [n \text{ heads}]} \left(\text{Attention}(XW_i^Q, XW_i^K, XW_i^V) \right) W^O \right)$ with Multi-Query Attention, there is just one W^K , W^V $\{ \text{displaystyle } W^K, W^V \}$, thus:

Section 4

/ first sublayer */* $z_d_copy \leftarrow \text{copy}(z_d)$ for each t in $1:\text{length}(z_d)$ do $z_d[t] \leftarrow \text{layer.layer_norm}(z_d[t])$ $z_d \leftarrow \text{layer.masked_multihead_attention}(z_d, z_d, z_d)$ for each t in $1:\text{length}(z_d)$ do $z_d[t] \leftarrow z_d[t] + z_d_copy[t]$

Section 5

In Multi-Token Prediction, a single forward pass creates a final embedding vector, which then is un-embedded into a token probability. However, that vector can then be further processed by another transformer block to predict the next token, and so on for arbitrarily many steps into the future. This trades off accuracy for speed, since each new token costs just one more transformer block, rather than the entire stack.

Section 6

Tokenizers, which convert text into tokens. Embedding layer, which converts tokens and positions of the tokens into vector representations. Transformer layers, which carry out repeated transformations on the vector representations, extracting more and more linguistic information. These consist of alternating attention and feedforward layers. There are two major types of transformer layers: encoder layers and decoder layers, with further variants. Un-embedding layer, which converts the final vector representations back to a probability distribution over the tokens. The following description follows exactly the transformer as described in the original paper. There are variants, described in the following section. By convention, we write all vectors as row vectors. For example, pushing a vector through a linear layer means multiplying it by a weight matrix on the right, as xW $\{ \text{displaystyle } xW \}$.

Section 7

Un-embedding An un-embedding layer is almost the reverse of an embedding layer. Whereas an embedding layer converts a token identifier into a vector, an un-embedding layer converts a vector into a probability distribution over tokens.

Section 8

where the softmax is applied over each of the rows of the matrix. The number of dimensions in a query vector is query size d_{query} $\{ \text{displaystyle } d_{\text{query}} \}$ and similarly for the key size d_{key} $\{ \text{displaystyle } d_{\text{key}} \}$ and value size d_{value} $\{ \text{displaystyle } d_{\text{value}} \}$. The output dimension of an attention head is its head dimension d_{head} $\{ \text{displaystyle } d_{\text{head}} \}$. The attention mechanism requires the following three equalities to hold: $d_{\text{seq, key}} = d_{\text{seq, value}}$, $d_{\text{query}} = d_{\text{key}}$, $d_{\text{value}} = d_{\text{head}}$ $\{ \text{displaystyle } d_{\text{seq, key}} = d_{\text{seq, value}}, d_{\text{query}} = d_{\text{key}}, d_{\text{value}} = d_{\text{head}} \}$ but is otherwise unconstrained. If the attention head is used in a self-attention fashion, then $X_{\text{query}} = X$

key = X value $\{\displaystyle X_{\text{query}}=X_{\text{key}}=X_{\text{value}}\}$. If the attention head is used in a cross-attention fashion, then usually $X_{\text{query}} \neq X_{\text{key}} = X_{\text{value}}$ $\{\displaystyle X_{\text{query}} \neq X_{\text{key}}=X_{\text{value}}\}$. It is theoretically possible for all three to be different, but that is rarely the case in practice.

Section 9

The final points of detail are the residual connections and layer normalization, (denoted as "LayerNorm", or "LN" in the following), which while conceptually unnecessary, are necessary for numerical stability and convergence. The residual connection, which is introduced to avoid vanishing gradient issues and stabilize the training process, can be expressed as follows: $y = F(x) + x$. The expression indicates that an output y is the sum of the transformation of input x ($F(x)$) and the input itself (x). Adding the input x can preserve the input information and avoid issues when the gradient of $F(x)$ is close to zero. Similarly to how the feedforward network modules are applied individually to each vector, the LayerNorm is also applied individually to each vector. There are two common conventions in use: the post-LN and the pre-LN convention. In the post-LN convention, the output of each sublayer is $\text{LayerNorm}(x + \text{Sublayer}(x))$ $\{\displaystyle \mathrm{LayerNorm}(x + \mathrm{Sublayer}(x))\}$ where $\text{Sublayer}(x)$ $\{\displaystyle \mathrm{Sublayer}(x)\}$ is the function implemented by the sublayer itself. In the pre-LN convention, the output of each sublayer is $x + \text{Sublayer}(\text{LayerNorm}(x))$ $\{\displaystyle x + \mathrm{Sublayer}(\mathrm{LayerNorm}(x))\}$ The original 2017 transformer used the post-LN convention. It was difficult to train and required careful hyperparameter tuning and a "warm-up" in learning rate, where it starts small and gradually increases. The pre-LN convention, proposed several times in 2018, was found to be easier to train, requiring no warm-up, leading to faster convergence.